

Name			
USN			
Date	Conduction	Record	Viva
Allotted Marks			
Obtained Marks			

EXPERIMENT 5

Implement Functions: Count – Sort – Limit – Skip – Aggregate using MongoDB

Aim: Implement Functions: Count – Sort – Limit – Skip – Aggregate using MongoDB

Theory:

MongoDB is a **NoSQL** database that stores data in a **flexible, JSON-like format** called **BSON** (Binary JSON). It is designed to handle large volumes of unstructured or semi-structured data with high performance and scalability.

Count - Returns the number of documents that match a query.

Sort- Orders the result set by one or more fields.

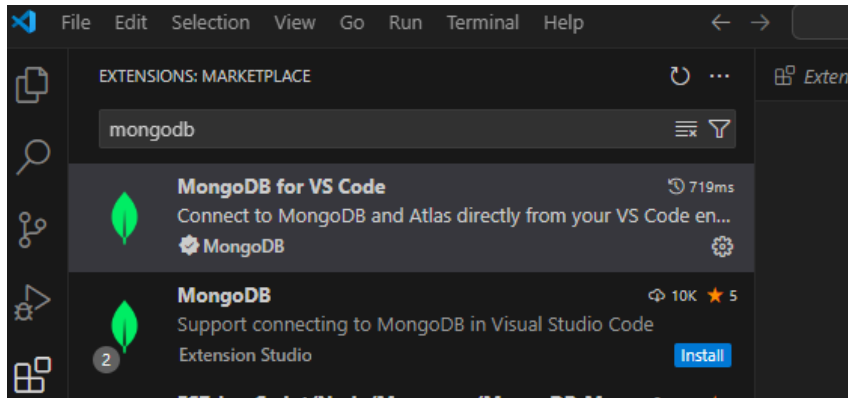
Limit- Restricts the number of documents returned.

Skip- Skips a specified number of documents in the result set.

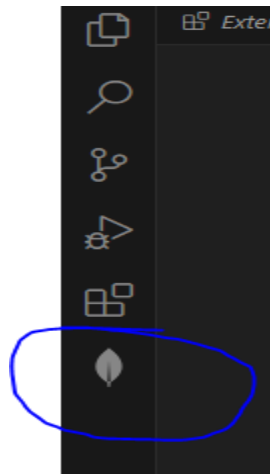
Aggregate- Processes data records and returns computed results. Used for **grouping, filtering, joining, projecting, and transforming** data.

Procedure:

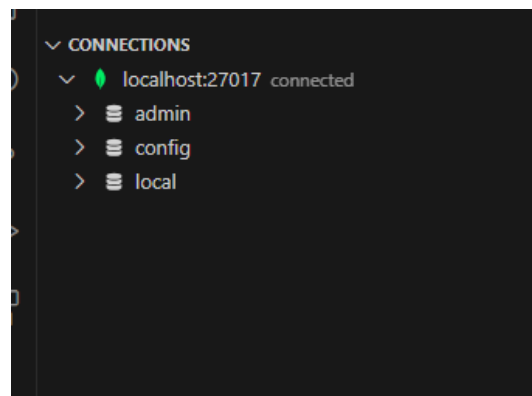
1. Go o VS code, click the extensions & search mangodb & install



2. Once it is installed, you get the symbol & to your left side



3. Go to add connections & press save & connect , Once the connections are done, you will get the below



4. Now, go to view & type `mongodb:launch mongodb shell` & press enter , you will get `test>` the powershell terminal.
If you get an error, install mongosh

5. Now perform count, limit, skip, sort & aggregate functions. Before proceeding to perform function, create a database

```
test> show dbs
admin    40.00 KiB
config  108.00 KiB
grades   48.00 KiB
local    72.00 KiB
```

Create a new database as mydb

```
test> use mydb
switched to db mydb
```

Now insert data into mydb, you can insert one by one or use the option insertmany({})

```
mydb> db.students.insert({"StudName": "Mabel mathews", "Grade": "VII", "Hobbies": "Baseball"})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('67f86726e5ee756de6b5f899') }
}
```

Once you have created the collections “students” in mydb. Check it

```
mydb> db.students.find()
[
  {
    _id: 1,
    StudName: ' Mabel mathews',
    Grade: 'VII',
    Hobbies: ' Baseball'
  },
  {
    _id: ObjectId('67f86726e5ee756de6b5f899'),
    'StudName': 'Mabel mathews',
    Grade: 'VII',
    Hobbies: 'Baseball'
  }
]
```

```
mydb> show collections
students
mydb> 
```

```
mydb> db.dropDatabase();
{ ok: 1, dropped: 'mydb' }
mydb> 
```

to drop data base

Now lets create a database for performing operations count, sort, limit, skip & aggregate

```
mydb> db.students.insert({"id":"1","Name":"Alice","Age":"24","Grade":"VII","Hobbies":"Baseball"})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('67f86d506b6b8bc924b5f899') }
}
mydb> db.students.insert({"id":"2","Name":"Bob","Age":"23","Grade":"VI","Hobbies":"Baseball"})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('67f86d7b6b6b8bc924b5f89a') }
}
mydb> db.students.insert({"id":"3","Name":"charlie","Age":"23","Grade":"VII","Hobbies":"chess"})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('67f86df96b6b8bc924b5f89b') }
}
mydb> db.students.insert({"id":"4","Name":"David","Age":"24","Grade":"VI","Hobbies":"Hockey"})
```

```
mydb> db.students.find().pretty();
[
  {
    _id: ObjectId('67f86d506b6b8bc924b5f899'),
    id: '1',
    Name: 'Alice',
    Age: '24',
    Grade: 'VII',
    Hobbies: 'Baseball'
  },
  {
    _id: ObjectId('67f86d7b6b6b8bc924b5f89a'),
    id: '2',
    Name: 'Bob',
    Age: '23',
    Grade: 'VI',
    Hobbies: 'Baseball'
  },
  {
    _id: ObjectId('67f86df96b6b8bc924b5f89b'),
    id: '3',
    Name: 'charlie',
    Age: '23',
    Grade: 'VII',
    Hobbies: 'chess'
  },
]
```

(i) COUNT OPERATION

```
mydb> db.students.count()
4
mydb> 
```

```
mydb> db.students.count({"Grade":"VII"})
2
mydb> |
```

(ii) LIMIT

```
mydb> db.students.find().limit(2).pretty()
[
  {
    _id: ObjectId('67f86d506b6b8bc924b5f899'),
    id: '1',
    Name: 'Alice',
    Age: '24',
    Grade: 'VII',
    Hobbies: 'Baseball'
  },
  {
    _id: ObjectId('67f86d7b6b6b8bc924b5f89a'),
    id: '2',
    Name: 'Bob',
    Age: '23',
    Grade: 'VI',
    Hobbies: 'Baseball'
  }
]
```

(iii) SORT

```
mydb> db.students.find().sort({"Grade":-1}).pretty()
[
  {
    _id: ObjectId('67f86d506b6b8bc924b5f899'),
    id: '1',
    Name: 'Alice',
    Age: '24',
    Grade: 'VII',
    Hobbies: 'Baseball'
  },
  {
    _id: ObjectId('67f86df96b6b8bc924b5f89b'),
    id: '3',
    Name: 'charlie',
    Age: '23',
    Grade: 'VII',
    Hobbies: 'chess'
  },
  {
    _id: ObjectId('67f86d7b6b6b8bc924b5f89a'),
    id: '2',
    Name: 'Bob',
    Age: '23',
    Grade: 'VI',
    Hobbies: 'Baseball'
  }
]
```

```
{
  _id: ObjectId('67f86e2c6b6b8bc924b5f89c'),
  id: '4',
  Name: 'David',
  Age: '24',
  Grade: 'VI',
  Hobbies: 'Hockey'
}
```

(iv) SKIP

```
mydb> db.students.find().skip(2).pretty()
[
  {
    _id: ObjectId('67f86df96b6b8bc924b5f89b'),
    id: '3',
    Name: 'charlie',
    Age: '23',
    Grade: 'VII',
    Hobbies: 'chess'
  },
  {
    _id: ObjectId('67f86e2c6b6b8bc924b5f89c'),
    id: '4',
    Name: 'David',
    Age: '24',
    Grade: 'VI',
    Hobbies: 'Hockey'
  }
]
```

COMBININIG all

```
mydb> db.students.find().sort({"Grade":-1}).skip(2).limit(2).pretty()
[
  {
    _id: ObjectId('67f86d7b6b6b8bc924b5f89a'),
    id: '2',
    Name: 'Bob',
    Age: '23',
    Grade: 'VI',
    Hobbies: 'Baseball'
  },
  {
    _id: ObjectId('67f86e2c6b6b8bc924b5f89c'),
    id: '4',
    Name: 'David',
    Age: '24',
    Grade: 'VI',
    Hobbies: 'Hockey'
  }
]
```

NOTE:

Create a database by name “products” which should include about 10 documents with the following entries

Id no.; product name; price; category; stock etc

Perform all the operations such as count, skip, sort, limit & aggregate

To calculate the aggregate for price

```
db.products.aggregate([
  {
    $group: {
      _id: "$category",
      averagePrice: { $avg: "$price" }
    }
  }
])
```

You can also check for stock

Conclusion:
